

Conjuntos em Python

COMP0496 – Programação A

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Imagine que você recebeu as matrículas de alunos de várias turmas e quer saber quais são **únicas**:

```
1  matriculas = [1001, 1002, 1003, 1001, 1004, 1002]
2
3  # Com lista: tedioso e lento  $O(n)$ 
4  unicas = []
5  for m in matriculas:
6      if m not in unicas:
7          unicas.append(m)
8  print(unicas)  # [1001, 1002, 1003, 1004]
9
10 # Com conjunto: direto
11 print(set(matriculas))  # {1001, 1002, 1003, 1004}
```

Conjunto

Coleção **sem duplicatas** e **sem ordem garantida**. Implementado com tabela hash — verificar pertencimento é $O(1)$.

Criando

- `s = {1, 2, 3}` (com elementos)
- `s = set()` (vazio)
- **Atenção:** `{}` cria **dicionário**, não conjunto!

Propriedades

- Sem duplicatas: `{1, 1, 2} = {1, 2}`
- Sem índice: `s[0]` gera `TypeError`
- Elementos devem ser *hasháveis* (`int`, `str`, `tuple`)

```
1  s = {10, 20, 30}
2  s.add(40)           # adicionar: {10, 20, 30, 40}
3  s.remove(20)        # remover (KeyError se ausente): {10, 30, 40}
4  s.discard(99)       # remover sem erro se ausente
5  print(10 in s)      # pertencimento O(1): True
6  print(99 in s)      # False
7  print(len(s))       # tamanho: 3
```

```
1  turmaA = {"Ana", "Bob", "Carlos"}
2  turmaB = {"Bob", "Diana", "Carlos"}
3
4  # Uniao: todos os alunos
5  print(turmaA | turmaB)      # {Ana, Bob, Carlos, Diana}
6
7  # Intersecao: alunos em ambas as turmas
8  print(turmaA & turmaB)     # {Bob, Carlos}
9
10 # Diferenca: so em turmaA
11 print(turmaA - turmaB)     # {Ana}
12
13 # Diferenca simetrica: em uma mas nao em ambas
14 print(turmaA ^ turmaB)     # {Ana, Diana}
```

```
1  comp0496 = {1001, 1002, 1003, 1004}
2  comp0497 = {1002, 1003, 1005, 1006}
3
4  # Alunos em ambas as disciplinas
5  dupla = comp0496 & comp0497
6  print("Dupla matricula:", dupla)      # {1002, 1003}
7
8  # Alunos so em COMP0496
9  so_496 = comp0496 - comp0497
10 print("So COMP0496:", so_496)         # {1001, 1004}
11
12 # Todos os alunos (sem repeticao)
13 todos = comp0496 | comp0497
14 print("Total de alunos:", len(todos)) # 6
```

Característica	<code>list</code>	<code>set</code>
Ordem garantida	Sim	Não
Duplicatas	Sim	Não
Acesso por índice	Sim	Não
<code>x in c</code> (busca)	$O(n)$	$O(1)$
Remoção de duplicatas	Tedioso	Automático
Operações de conjuntos	Não	Sim

Regra prática: use `set` quando precisar de pertencimento rápido ou deduplicação; use `list` quando a ordem ou as duplicatas importam.

- **Conjunto** armazena elementos únicos sem ordem garantida.
- `{}` vazio cria `dict` — use `set()` para conjunto vazio.
- Operadores `|`, `&`, `-`, `^` realizam união, interseção, diferença e diferença simétrica.
- Pertencimento `x in s` é $O(1)$ — muito mais rápido que em listas.